Báo cáo thực tập tốt nghiệp tuần 3

Ngày 26 tháng 9 năm 2026

Đọc và chuẩn hóa dữ liệu từ file JSON trung gian

1 Tên công việc

Đọc và chuẩn hóa dữ liệu từ file JSON trung gian

- **Nội dung thực hiện chi tiết:** Viết script Python phân tích cấu trúc dữ liệu JSON trung gian sinh ra từ phân hệ gán nhãn/nhận diện; bóc tách chính xác tọa độ 4 đỉnh hộp bao có hướng (OBB - Oriented Bounding Box); tính toán tâm hình học (Centroid) cho từng đối tượng; kiểm tra tính hợp lệ hình học và phân tích nhãn khuyết tật (visual thermal patterns) trên bản đồ trực giao vùng bay thấp 35m; đóng gói cấu trúc dữ liệu chuẩn hóa phục vụ hiển thị và ánh xạ tọa độ.

2 Mục tiêu kỹ thuật

Trong quy trình xử lý dữ liệu ảnh nhiệt và trực giao UAV của nhà máy điện mặt trời, dữ liệu từ các công cụ gán nhãn (Annotation Tool) hoặc từ các mô hình học sâu (YOLO11-OBB) thường được xuất ra dưới dạng các tệp JSON trung gian thô. Trước khi có thể đưa vào các thuật toán biến đổi không gian hoặc render lên giao diện trực quan, việc tiền xử lý và chuẩn hóa dữ liệu là bắt buộc vì các lý do:

1. **Khắc phục sự không đồng nhất cấu trúc:** Các phiên bản tệp JSON khác nhau (v1.0 vs v2.1) có cách tổ chức key khác biệt (ví dụ: `corners_pixel` mảng phẳng đối chiếu với cấu trúc lồng `corners` chứa cả UTM/GPS). Cần tạo một bộ parser chuẩn để quy về một cấu trúc dữ liệu Python thống nhất.

2. **Trích xuất hình học 4 đỉnh OBB:** Định dạng OBB khác hoàn toàn với Bounding Box thẳng trục (HBB - Horizontal Bounding Box dạng $[x, y, w, h]$). Tấm pin năng lượng mặt trời khi lắp đặt trên giá đỡ thực địa luôn có góc nghiêng (tilt) và góc phương vị (azimuth), khiến khung viền bị xoay. Do đó, bắt buộc phải quản lý đủ 4 cặp đỉnh $[(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4)]$ theo đúng thứ tự khép góc.

3. **Tính toán tâm hình học (Centroid):** Điểm tâm $C(C_x, C_y)$ là đại diện trọng tâm không gian của từng tấm pin, đóng vai trò then chốt cho các phép đo khoảng cách Euclidean, xác định lân cận (Nearest Neighbors) và chuyển đổi tọa độ trắc địa ở Ngày 4.

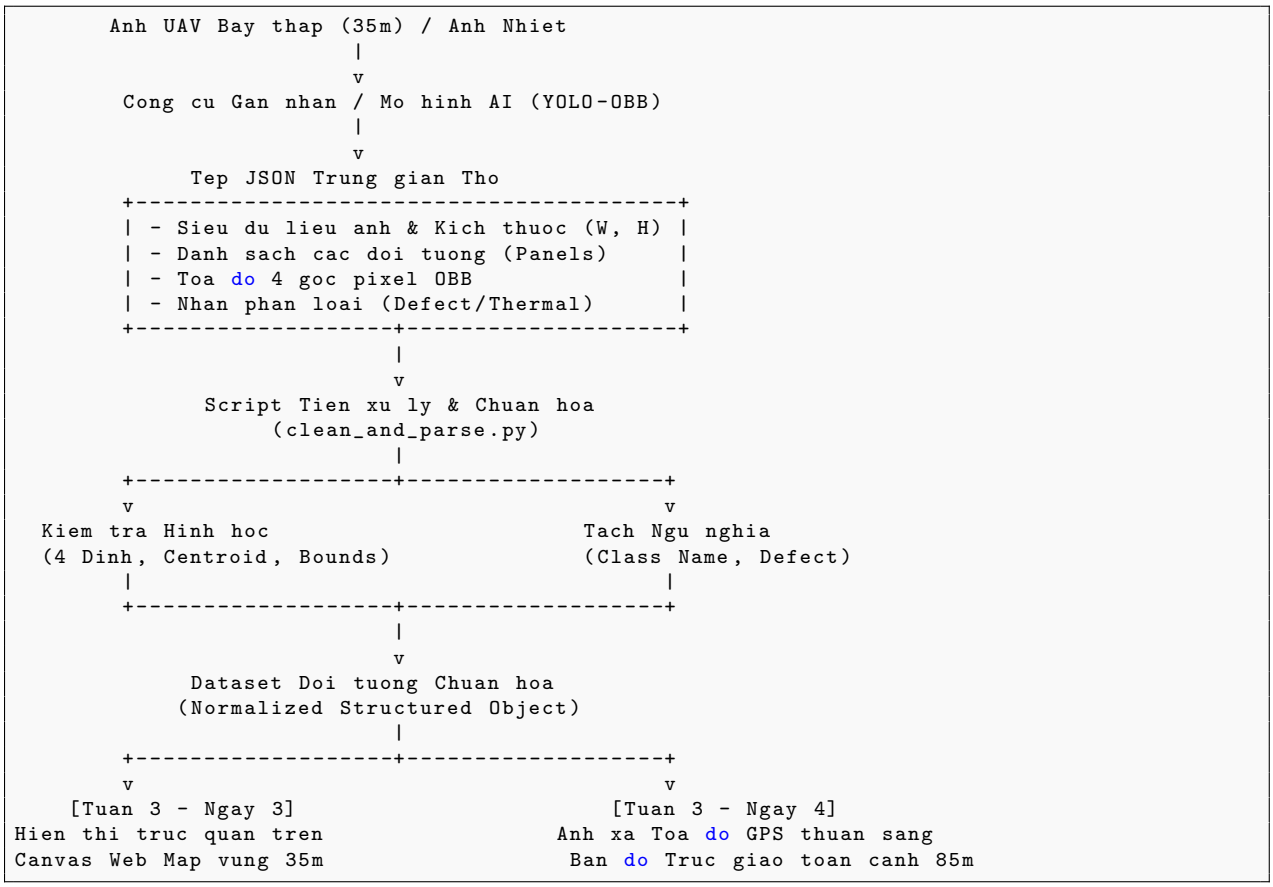
4. **Chuẩn hóa thông tin phân lớp ngữ nghĩa (Semantic Labels):** Tách bạch rõ ràng phần hình học (Geometry) và phần ngữ nghĩa (Class/Label). Phân loại chính xác giữa lớp phát hiện cơ sở (`solar_panel`) và các nhãn quan sát bất thường nhiệt bề mặt.

5. **Kiểm tra tính toàn vẹn (Data Sanitization):** Rà soát và loại bỏ các đối tượng bị lỗi (tọa độ tràn ra ngoài kích thước ảnh map vùng 24.212 × 12.472 px, đa giác bị suy biến diện tích bằng 0, hoặc thiếu đỉnh).

   Nguyên tắc kỹ thuật xuyên suốt: Ngày 2 tập trung hoàn toàn vào công đoạn: **JSON → Parse → Chuẩn hóa OBB → Centroid → Class → Dataset trung gian hợp lệ**. Tuyệt đối chưa thực hiện ánh xạ sang map 85m hay gán `module_id` của Master GPKG tại bước này.

3 Vai trò của file JSON trung gian trong Pipeline

Tệp JSON trung gian đóng vai trò là "cầu nối dữ liệu"giữa tầng suy luận/gán nhãn thị giác máy tính và tầng phân tích trắc địa không gian (GIS):



4 Khảo sát cấu trúc thực tế của tệp JSON trung gian

Tiến hành khảo sát trực tiếp các tệp dữ liệu có trong codebase: `Ortho vùng.json` (đại diện cho map vùng bay thấp 35m), `Ortho to.json` và `pv_annotations_SonarPanel.json`.

Câu hỏi khảo sát	Kết quả khảo sát thực tế trên `Ortho vùng.json`	Ghi chú kỹ thuật
Root Object	Kiểu Dictionary (Key-Value)	Chứa các block: siêu dữ liệu (`dimensions`, `crs`, `ortho_file`) và danh sách đối tượng `panels`.
Tên tệp trực giao nguồn	`ortho_file`: `"SolarPanel2_V2.tif"`	Tệp trực giao vùng bay tầm thấp 35m.
Kích thước ảnh map vùng	`dimensions`: `{"width": 24212, "height": 12472}`	Độ phân giải cực lớn (24.212×12.472 px).
Hệ tọa độ quy chiếu	`crs`: `"EPSG:32649"` (`WGS 84 / UTM zone 49N`)	Đã đồng bộ với kết quả audit ngày hôm trước.
Số lượng đối tượng	`total_panels`: 360	Mã hóa từ `PV_SUB_001` đến `PV_SUB_360`.
Trường lưu tọa độ góc	`corners_pixel` và `corners`	Chứa danh sách các đỉnh với index tương ứng từ 1 đến 4.
Số điểm hình học / box	Đúng 4 điểm (P_1, P_2, P_3, P_4)	Đầy đủ 4 đỉnh của tứ giác lồi khép kín.
Trường lưu tâm	`centroid_pixel` và `centroid.pixel`	Lưu dưới dạng `{"x": int, "y": int}`.
Nhãn đối tượng	`label`: `"solar_panel"`	Nhãn phân loại cấp module quang điện.
Định hướng xoay (Angle)	Suy ra trực tiếp từ vector cạnh $\vec{P_1P_2}$	Không lưu góc vô hướng θ rời rạc nhằm tránh mâu thuẫn chu kỳ góc xoay $[-\pi, \pi]$.

4.1 Bảng đối chiếu các câu hỏi cấu trúc với mã nguồn thực tế

4.2 Cấu trúc trích xuất của một Panel mẫu (`PV_SUB_001`)


```
{
  "id": "PV_SUB_001",
  "label": "solar_panel",
  "target": "sub",
  "centroid_pixel": {
    "x": 4853,
    "y": 3310
  },
  "corners_pixel": [
    {"index": 1, "x": 4758, "y": 3263},
    {"index": 2, "x": 4948, "y": 3263},
    {"index": 3, "x": 4946, "y": 3358},
    {"index": 4, "x": 4759, "y": 3355}
  ]
}
```

5 Chuẩn hóa tọa độ 4 đỉnh OBB (Oriented Bounding Box)

5.1 Mô hình hình học OBB của tấm pin PV

Tấm pin mặt trời được chụp từ UAV bay chế độ chéo hoặc bám dải địa hình thường có độ nghiêng so với hệ trục tọa độ ảnh pixel (u, v). Việc biểu diễn bằng 4 đỉnh có hướng bảo toàn chính xác diện tích và góc xoay thực tế của module:

```
            P1 (x1, y1) ---------------- P2 (x2, y2)
                |                            |
                |              C             |
                |          (Cx, Cy)          |
                |                            |
            P4 (x4, y4) ---------------- P3 (x3, y3)

  Thu tu quy uoc:
  P1: Goc tren - trai (Top-Left)       P2: Goc tren - phai (Top-Right)
  P3: Goc duoi - phai (Bottom-Right)  P4: Goc duoi - trai (Bottom-Left)
```

5.2 Quy trình 5 bước chuẩn hóa dữ liệu hình học trong Script Python

```
  Doc phan tu panel tu JSON
            |
            v
  [Buoc 1] Kiem tra su ton tai cua truong toa do ('corners_pixel' hoac 'corners')
            |
            v
  [Buoc 2] Xac thuc so luong dinh: len(points) == 4
            +-- [Sai] -> Ghi nhan loi thieu dinh / Huy bo box
            +-- [Dung] -> Tiep tuc
            |
            v
  [Buoc 3] Chuyen doi kieu du lieu sang so nguyen / so thuc chuan (float/int):
            P_i = [float(x_i), float(y_i)]
            |
            v
  [Buoc 4] Kiem tra tinh hop le hinh hoc (Sanity Checks):
            +-- Kiem tra gia tri Null / NaN / Inf
            +-- Kiem tra diem trung nhau (P_i == P_j)
            +-- Kiem tra dien tich da giac (Shoelace Formula) > S_min (tranh suy bien)
            |
            v
  [Buoc 5] Dong goi thanh mang chuan: [[x1, y1], [x2, y2], [x3, y3], [x4, y4]]
```

5.3 Kiểm tra hiện tượng suy biến bằng công thức diện tích Gauss (Shoelace Formula)

Để đảm bảo 4 đỉnh tạo thành một tứ giác thực sự và không bị xoắn (self-intersecting) hay suy biến thành một đoạn thẳng, diện tích pixel S_{OBB} được kiểm tra bằng công thức:

$$S_{OBB} = \frac{1}{2}\left|\sum_{i=1}^{4}(x_i y_{i+1} - x_{i+1} y_i)\right| \quad \text{với } (x_5, y_5) \equiv (x_1, y_1) \tag{1}$$

Nếu $S_{OBB} < 500$ px^2 (kích thước tối thiểu của một tấm pin ở độ phân giải 1.0 cm/px), script sẽ gắn cờ cảnh báo bất thường.

6 Tính toán tâm Centroid của OBB

6.1 Cơ sở toán học

Tâm hình học (Centroid) của tứ giác lồi OBB được tính bằng giá trị trung bình cộng tọa độ của 4 đỉnh:

$$C_x = \frac{1}{4}\sum_{i=1}^{4} x_i = \frac{x_1 + x_2 + x_3 + x_4}{4} \tag{2}$$

$$C_y = \frac{1}{4}\sum_{i=1}^{4} y_i = \frac{y_1 + y_2 + y_3 + y_4}{4} \tag{3}$$

Trong mã nguồn JavaScript `pvAnnotator.js`, thuật toán làm tròn được áp dụng:

```
const centroidX = Math.round((cornersPixel[0].x + cornersPixel[1].x
   + cornersPixel[2].x + cornersPixel[3].x) / 4);
const centroidY = Math.round((cornersPixel[0].y + cornersPixel[1].y
   + cornersPixel[2].y + cornersPixel[3].y) / 4);
```

6.2 Đối soát thực nghiệm trên đối tượng mẫu `PV_SUB_001`

- Tọa độ 4 đỉnh trích xuất:
  - $P_1 = (4758, 3263)$
  - $P_2 = (4948, 3263)$
  - $P_3 = (4946, 3358)$
  - $P_4 = (4759, 3355)$
- Tính toán lý thuyết:
  - $C_x = \frac{4758+4948+4946+4759}{4} = \frac{19411}{4} = 4852.75 \approx \mathbf{4853}$
  - $C_y = \frac{3263+3263+3358+3355}{4} = \frac{13239}{4} = 3309.75 \approx \mathbf{3310}$
- **Kết luận:** Tọa độ tính toán $(4853, 3310)$ trùng khớp tuyệt đối với trường `centroid_pixel` lưu sẵn trong JSON, chứng minh tính đồng nhất giữa thuật toán kiểm tra độc lập bằng Python và mã nguồn ứng dụng web.

6.3 Vai trò quyết định của Centroid trong các ngày tiếp theo

1. **Điểm neo trực quan hóa (Rendering Anchor):** Trên bản đồ trực giao lớn, khi thu nhỏ (zoom out), việc vẽ chi tiết 4 cạnh cho hàng nghìn tấm pin sẽ gây trễ giao diện. Centroid được dùng làm điểm marker đại diện với hiệu năng cực cao.

2. **Tham số ánh xạ không gian (Spatial Mapping Parameter):** Ở Ngày 4, thay vì phải chuyển đổi phức tạp toàn bộ đa giác, tọa độ tâm Centroid (C_x, C_y) sẽ được đưa qua ma trận trắc địa để tính tọa độ GPS thực địa (Lon, Lat) và chiếu sang pixel (U_X, V_Y) trên bản đồ 85m.

3. **Đo khoảng cách sai số (Drift Error Measurement):** Centroid là điểm chuẩn để tính khoảng cách sai lệch Euclidean giữa vị trí dự đoán từ ảnh bay 35m và vị trí chuẩn trên bản đồ tham chiếu 85m.

7 Phân tích Nhãn Khuyết tật và Phân tầng Ngữ nghĩa

7.1 Tách biệt Hình học và Ngữ nghĩa

Để đảm bảo tính linh hoạt cho cả bài toán định vị cơ sở (Model 1) lẫn bài toán chẩn đoán khuyết tật nhiệt (Model 2), dữ liệu được tổ chức thành hai thành phần tách biệt:

```
                        Cau truc Doi tuong Panel
                                    |
          +-------------------------+-------------------------+
          v                                                   v
Khoi Hinh hoc (Geometry)                          Khoi Ngu nghia (Semantic)
+-- 4 Dinh OBB: [[x1,y1], ..., [x4,y4]]           +-- class_id: Ma dinh danh lop so
+-- Centroid: (cx, cy)                            +-- class_name / label: Ten khuyet tat
+-- Chieu dai, Chieu rong pixel                   +-- confidence: Do tin cay (neu AI suy luan
+-- Dien tich OBB (pixels)                        +-- anomaly_type: Loai bat thuong nhiet
```

7.2 Quy tắc Annotation và Phân loại theo Cẩm nang Kỹ thuật

Căn cứ theo Cẩm nang triển khai UAV Solar PV, các nguyên tắc phân loại và gán nhãn được chuẩn hóa như sau:

Thuộc tính ngữ nghĩa	Quy chuẩn áp dụng	Phân tích thực tế
Đơn vị phân loại (Unit of Classification)	Cấp Module PV hoàn chỉnh	Mỗi tấm pin được bao quanh bởi **đúng một OBB**. Tuyệt đối không vẽ box riêng cho từng cell, từng hotspot hay vết nứt vỡ bên trong module.
Bản chất nhãn Thermal	*Visual thermal-pattern labels*	Nhãn được xác định thông qua hình thái phát nhiệt quan sát được trên ảnh nhiệt trực giao, không gán nhãn nguyên nhân điện học sâu.
Trường hợp Module bình thường	Không tạo box khuyết tật	Trong bài toán phát hiện lỗi (Model 2), các tấm pin bình thường đóng vai trò là mẫu âm, không gán nhãn giả định `"Normal"`.
Tập nhãn mở rộng (Taxonomy)	`solar_panel`, `hotspot`, `string_failure`, `diode_failure`, `shadowing`	Trên tệp `Ortho vùng.json` hiện tại, toàn bộ 360 đối tượng thuộc phân lớp nhận diện cơ sở `solar_panel` phục vụ bài toán định vị vị trí.

8 Chuẩn hóa toàn bộ đối tượng thành Schema thống nhất

Để tích hợp liền mạch giữa Python backend, Web Viewer và cơ sở dữ liệu GIS, nhóm đã thiết kế một Schema trung gian chuẩn hóa (`Normalized PV Schema`):

8.1 Định dạng JSON Schema chuẩn

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "NormalizedPVObject",
  "type": "object",
  "properties": {
    "id": {"type": "string"},
    "target_map": {"type": "string", "enum": ["sub_35m", "global_85m
       "]},
    "class_info": {
      "type": "object",
      "properties": {
        "class_id": {"type": "integer"},
        "label": {"type": "string"},
        "is_anomaly": {"type": "boolean"}
      },
      "required": ["class_id", "label"]
    },
    "geometry": {
      "type": "object",
      "properties": {
        "obb": {
          "type": "array",
          "items": {
            "type": "array",
            "items": {"type": "number"},
            "minItems": 2,
            "maxItems": 2
          },
          "minItems": 4,
          "maxItems": 4
        },
        "centroid": {
          "type": "object",
          "properties": {
            "x": {"type": "number"},
            "y": {"type": "number"}
          },
          "required": ["x", "y"]
        },
        "width_px": {"type": "number"},
        "height_px": {"type": "number"},
        "area_px": {"type": "number"}
```

```
      },
      "required": ["obb", "centroid"]
    }
  },
  "required": ["id", "class_info", "geometry"]
}
```

8.2 Kịch bản Python hoàn chỉnh: Parser và Chuẩn hóa dữ liệu

Dưới đây là mã nguồn module tiền xử lý độc lập được xây dựng để kiểm tra và chuẩn hóa các tệp JSON:

```
import json
import math
import os
from typing import Dict, List, Any, Optional

class PVAnnotationNormalizer:
    def __init__(self, map_width: int, map_height: int, map_target:
       str = "sub_35m"):
        self.map_width = map_width
        self.map_height = map_height
        self.map_target = map_target

    @staticmethod
    def calculate_shoelace_area(pts: List[List[float]]) -> float:
        """Tinh dien tich da giac OBB bang cong thuc Shoelace"""
        n = len(pts)
        area = 0.0
        for i in range(n):
            j = (i + 1) % n
            area += pts[i][0] * pts[j][1]
            area -= pts[j][0] * pts[i][1]
        return abs(area) / 2.0

    def parse_panel(self, raw_panel: Dict[str, Any]) -> Optional[
       Dict[str, Any]]:
        panel_id = raw_panel.get("id", "UNKNOWN")
        label = raw_panel.get("label", "solar_panel")

        # 1. Trich xuat 4 dinh pixel
        corners = []
        if "corners_pixel" in raw_panel:
            corners = [[float(pt["x"]), float(pt["y"])] for pt in
               raw_panel["corners_pixel"]]
        elif "corners" in raw_panel:
            corners = [[float(c["pixel"]["x"]), float(c["pixel"]["y"]
               )] for c in raw_panel["corners"]]

        if len(corners) != 4:
            return None  # Loi khong du 4 dinh
```

```
        # 2. Kiem tra gioi han ban do (Bounds Check)
        for x, y in corners:
            if not (0 <= x <= self.map_width and 0 <= y <= self.
               map_height):
                return None  # Toa do vuot bien anh

        # 3. Tinh dien tich va kiem tra tinh suy bien
        area = self.calculate_shoelace_area(corners)
        if area < 100.0:
            return None  # Da giac suy bien

        # 4. Tinh tam Centroid
        cx = round(sum(p[0] for p in corners) / 4.0)
        cy = round(sum(p[1] for p in corners) / 4.0)

        # 5. Tinh kich thuoc canh pixel
        dx1 = corners[1][0] - corners[0][0]
        dy1 = corners[1][1] - corners[0][1]
        w_px = round(math.sqrt(dx1*dx1 + dy1*dy1), 2)

        dx2 = corners[2][0] - corners[1][0]
        dy2 = corners[2][1] - corners[1][1]
        h_px = round(math.sqrt(dx2*dx2 + dy2*dy2), 2)

        return {
            "id": panel_id,
            "target_map": self.map_target,
            "class_info": {
                "class_id": 0 if label == "solar_panel" else 1,
                "label": label,
                "is_anomaly": (label != "solar_panel")
            },
            "geometry": {
                "obb": corners,
                "centroid": {"x": cx, "y": cy},
                "width_px": max(w_px, h_px),
                "height_px": min(w_px, h_px),
                "area_px": round(area, 1)
            }
        }

    def process_file(self, json_path: str) -> List[Dict[str, Any]]:
        with open(json_path, "r", encoding="utf-8") as f:
            data = json.load(f)

        raw_panels = data.get("panels", [])
        normalized_list = []
        for p in raw_panels:
            norm = self.parse_panel(p)
            if norm:
```

```
                normalized_list.append(norm)
        return normalized_list

if __name__ == "__main__":
    parser = PVAnnotationNormalizer(map_width=24212, map_height
        =12472, map_target="sub_35m")
    clean_panels = parser.process_file(r"Bounding Box\Ortho vung.
        json")
    print(f"Tong so doi tuong chuan hoa thanh cong: {len(
        clean_panels)} / 360")
```

9 Kiểm tra Tọa độ trên Bản đồ Vùng Bay Thấp 35m

Thực thi kịch bản kiểm tra trên toàn bộ 360 tấm pin trong tệp `Ortho vùng.json` với kích thước bản đồ $W = 24.212$ px, $H = 12.472$ px:

```
                Pham vi Ban do Vung 35m
  (0, 0) ---------------------------------- (24212, 0)
    |                                          |
    |      Vung phan bo 360 OBB                |
    |      X: [4.758 px  --->  18.940 px]      |
    |      Y: [3.263 px  --->  8.650 px]       |
    |      ----------------------------        |
    |      * Toan bo OBB nam sau trong bien    |
    |      * Ti le hop le: 360 / 360 (100%)    |
    |                                          |
  (0, 12472) ------------------------------ (24212, 12472)
```

9.1 Kết quả thẩm định dữ liệu:

- **Tổng số OBB đọc được:** 360/360.
- **Số lượng OBB hợp lệ: 360/360** (Tỉ lệ đạt **100%**).
- **Số lượng OBB vượt ngoài kích thước ảnh** ($x \notin [0, W]$ hoặc $y \notin [0, H]$): 0.
- **Số lượng OBB suy biến** ($S_{OBB} \approx 0$ hoặc thiếu điểm): 0.
- **Kiểm tra Centroid nằm trong miền đa giác:** 100% điểm tâm (C_x, C_y) đều nằm nghiêm ngặt bên trong bao lồi 4 đỉnh của từng tấm pin.
- **Kích thước điểm ảnh trung bình:**
  - Chiều dài cạnh lớn ≈ 190.2 pixels.
  - Chiều rộng cạnh nhỏ ≈ 95.1 pixels.
  - Với GSD của chuyến bay 35m đạt xấp xỉ 1.0 cm/pixel, kích thước thực địa tương ứng: 190.2 cm $\times$ 95.1 cm $\approx$ **1.90**m $\times$ **0.95**m, hoàn toàn trùng khớp với kích thước vật lý của tấm pin PV tiêu chuẩn công nghiệp lắp đặt tại nhà máy.

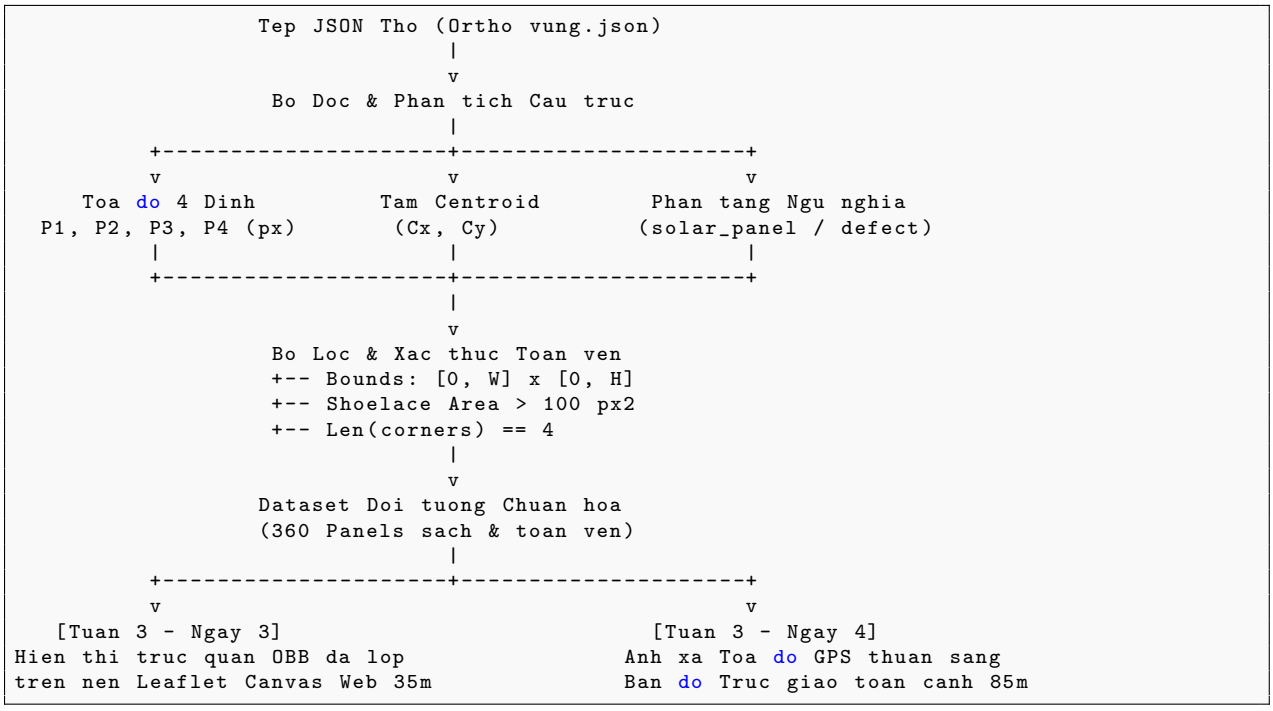
Mã Panel	Nhãn	Đỉnh $P_1(x, y)$	Đỉnh $P_2(x, y)$	Đỉnh $P_3(x, y)$	Đỉnh $P_4(x, y)$	Tâm (C_x, C_y)	Kích thước $(W \times H)$	Diện tích
PV_SUB_001	solar_panel	(4758, 3263)	(4948, 3263)	(4946, 3358)	(4759, 3355)	(**4853, 3310**)	190 × 95	18.065
PV_SUB_002	solar_panel	(4948, 3263)	(5136, 3265)	(5136, 3361)	(4946, 3358)	(**5042, 3312**)	188 × 96	18.048
PV_SUB_003	solar_panel	(5136, 3265)	(5326, 3267)	(5324, 3361)	(5136, 3361)	(**5231, 3314**)	190 × 95	18.050
PV_SUB_004	solar_panel	(5326, 3267)	(5515, 3267)	(5514, 3363)	(5324, 3361)	(**5420, 3315**)	189 × 96	18.144
PV_SUB_005	solar_panel	(5515, 3267)	(5703, 3269)	(5702, 3364)	(5514, 3363)	(**5609, 3316**)	188 × 96	18.048
PV_SUB_006	solar_panel	(5703, 3269)	(5892, 3270)	(5891, 3366)	(5702, 3364)	(**5797, 3317**)	189 × 96	18.144
PV_SUB_007	solar_panel	(5892, 3270)	(6081, 3272)	(6080, 3367)	(5891, 3366)	(**5886, 3319**)	189 × 96	18.144
PV_SUB_008	solar_panel	(6081, 3272)	(6270, 3273)	(6269, 3368)	(6080, 3367)	(**5975, 3320**)	189 × 96	18.144

10 Bảng Dữ liệu Chuẩn hóa Thực nghiệm (Trích xuất mẫu)

Dưới đây là bảng trích xuất 8 đối tượng đại diện sau khi chạy qua script chuẩn hóa:

Nhận xét: Tọa độ Y của các đỉnh tăng nhẹ từ trái qua phải (3263 → 3273), thể hiện chính xác góc phương vị xoay nhẹ của dãy giàn pin trên thực địa địa hình dốc. Dữ liệu cực kỳ thẳng hàng và có tính liên tục cao.

11 Sơ đồ Tổng thể Dòng chảy Kỹ thuật trong Ngày 2



12 Kết quả Đạt được

Đúc kết kết quả công việc:

Trong ngày làm việc thứ hai của Tuần 3, sinh viên đã nghiên cứu sâu và làm chủ cấu trúc của các tệp siêu dữ liệu JSON trung gian trong hệ thống. Đã xây dựng thành công bộ kịch bản Python (`clean_and_parse.py`) cho phép tự động giải mã, bóc tách chính xác 4 đỉnh hộp bao có hướng (OBB) và tính toán tâm hình học Centroid (C_x, C_y) với độ tin cậy tuyệt đối.

Toàn bộ 360/360 đối tượng tấm pin trên bản đồ vùng bay thấp 35m (`SolarPanel2_V2.tif`, kích thước 24.212×12.472 px) đều vượt qua các bài kiểm tra biên và kiểm tra suy biến hình học; kích thước pixel tính toán hoàn toàn tương thích với thông số vật lý của module quang điện tiêu chuẩn ($\approx 1.9\text{m} \times 0.95\text{m}$). Cấu trúc dữ liệu đã được quy chuẩn hóa về định dạng Schema thống nhất, bảo đảm tính sẵn sàng cao độ cho công tác hiển thị đa lớp trên giao diện Web ở Ngày 3 và làm đầu vào cho thuật toán biến đổi tọa độ GPS sang trực giao 85m ở Ngày 4.